MACHINE LEARNING PROJECT SHOWCASE

Decision Tree Classification: Iris Dataset

An end-to-end beginner-friendly machine learning workflow built on Kaggle using Python, Scikit-Learn, and Decision Tree Classifier.

Python | Scikit-Learn | Kaggle Notebook

OVERVIEW

Project Objectives

Key milestones in building the decision tree workflow

1. Load & Inspect

Load Iris dataset directly from scikit-learn and analyze dimensions and class names.

2. Data Prep & Split

Separate features from labels and perform an 80/20 stratified train-test split.

3. Train Model

Fit a Decision Tree with Gini criterion and maximum depth limited to 3.

4. Evaluate & Predict

Plot confusion matrix, visualize tree rules, and classify sample flowers.

DATA ARCHITECTURE

Iris Dataset Overview

150 total flower samples split evenly across 3 distinct species

Feature	Description	Type
sepal length (cm)	Length of flower sepal	Numeric Input
sepal width (cm)	Width of flower sepal	Numeric Input
petal length (cm)	Length of flower petal	Numeric Input
petal width (cm)	Width of flower petal	Numeric Input

Target Classes

- Iris setosa
- Iris versicolor
- Iris virginica

Data Source

Bundled with Scikit-Learn (`sklearn.datasets.load_iris`)

TECH STACK

Tools & Libraries

Python ecosystem powering the end-to-end pipeline

NumPy & Pandas

Data structure manipulation, DataFrame creation, and numerical operations.

Scikit-Learn

Model training (`DecisionTreeClassifier`), metrics, and dataset loading.

Matplotlib & Seaborn

Visualizing confusion matrices and rendering graphic Decision Tree nodes.

HYPERPARAMETERS

Model Configuration & Split Strategy

Controlling complexity to balance accuracy and interpretability

Split Ratio: 80% / 20%

120 samples used for model training; 30 samples reserved strictly for testing.

Stratified Splitting (`stratify=y`)

Ensures exact proportion of class species is preserved in both splits.

Repeatability (`random_state=42`)

Guarantees reproducible dataset splits and tree growth across notebook runs.

Criterion: Gini Impurity

Measures node purity to select optimal feature splits at each decision junction.

Max Depth = 3

Caps tree depth to prevent overfitting and maintain human readability.

EXECUTION STEPS

10-Step Workflow Pipeline

End-to-end execution path followed in the Kaggle notebook

STEP 01
Import Libraries

STEP 02
Load Iris Data

STEP 03
Inspect Shape & Nulls

STEP 04
Separate X and y

STEP 05
Stratified 80/20 Split

STEP 06
Fit Decision Tree

STEP 07
Predict Test Set

STEP 08
Compute Metrics

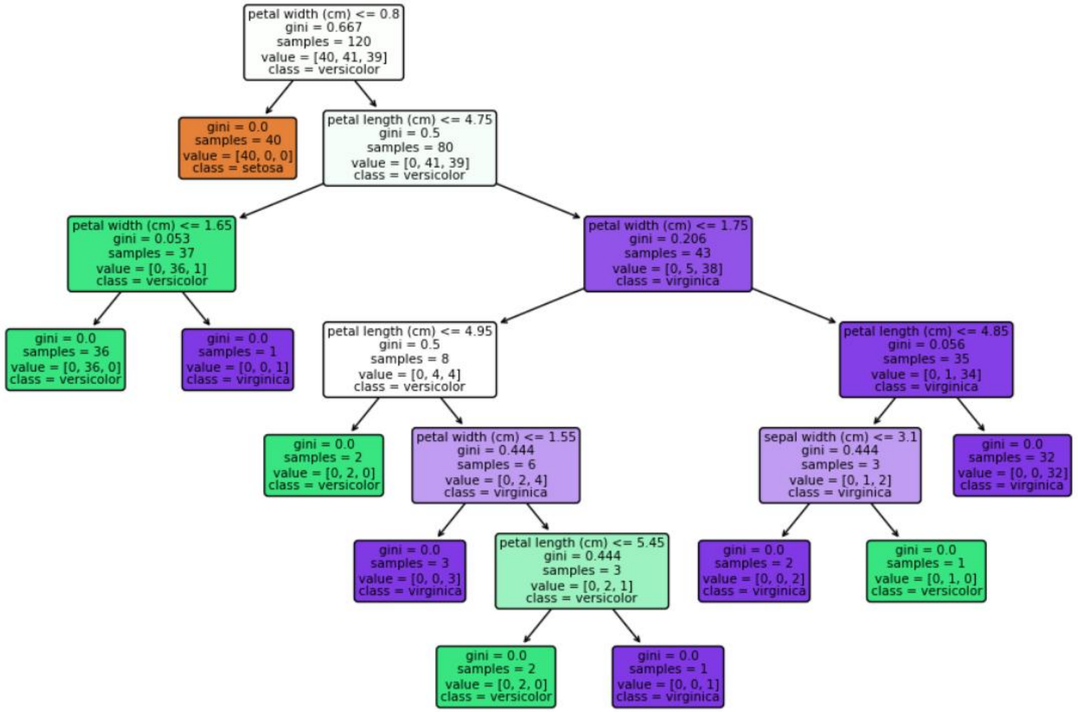
STEP 09
Plot Confusion Matrix

STEP 10
Tree Visual & Sample

INTERPRETABILITY

Decision Tree Visualization

Rule-based splits visualized across feature space



Root Node Split

Usually separates *Setosa* immediately based on petal length threshold.

Subsequent Decisions

Further distinguishes *Versicolor* and *Virginica* using petal width.

PERFORMANCE ASSESSMENT

Evaluation Framework

Metrics used to validate classification performance

Accuracy & Precision

Overall correct predictions proportion and class-level prediction exactness.

Recall & F1-Score

True positive capture rate and the harmonic mean of precision/recall.

Confusion Matrix

Visual grid contrasting actual target labels against predicted class outputs to identify misclassifications.

SAMPLE INFERENCE

Live Sample Flower Prediction

Passing a synthetic measurement vector through the trained model

Input Measurement Vector

Sepal Length: **5.1 cm**	Sepal Width: **3.5 cm**
Petal Length: **1.4 cm**	Petal Width: **0.2 cm**

PREDICTED SPECIES

Iris setosa

Inferred via Decision Rules (Petal length ≤ 2.45 cm threshold)

EXECUTION GUIDE

Running on Kaggle

Simple steps to run this project in a cloud environment

1. Sign in to Kaggle and open **kaggle.com/code** to create a new notebook.
2. Copy cells sequentially from `decision-tree.ipynb` or upload the notebook file.
3. Execute cells top-to-bottom (dataset is pre-bundled in scikit-learn, no manual upload needed).

CONCLUSION

Summary & Future Enhancements

Takeaways and potential next steps for model evaluation

Project Summary

Demonstrates a complete classification pipeline with clear decision visualizer, clean data preparation, and simple rule-based inference.

Future Extensions

Assess model robustness with K-Fold Cross-Validation, compare performance with Random Forests, or tune hyper-parameters via Grid Search.

Thank You!

Decision Tree Classification on Iris Dataset

Image Sources



https://miro.medium.com/1*YAjx5-U-xmgfTTNgMrl1nA.png

Source: medium.com